Canny Edge Detection

December 16, 2025

1 Overview

In Assignment 1, you may have noticed that your output edge maps were not exactly similar to the given sample output. Some common discrepancies included thicker edges and missing or broken edges compared to the reference image. These issues arise because the basic edge detection pipeline is often insufficient for producing thin, well-localized, and continuous edges. These problems can be addressed by adding three additional steps to the edge detection pipeline: Non-Maximum Suppression (NMS), double thresholding, and edge hysteresis. These steps are collectively used in the popular Canny edge detection algorithm to significantly improve edge quality.
The pipeline followed in Assignment 1 was:
Grayscale → Smoothing → Edge detection filters (Gx, Gy) → Gradient magnitude → Thresholding
In contrast, the Canny edge detection pipeline is:
Grayscale → Smoothing → Edge detection filters → Gradient magnitude & direction → Non-Maximum Suppression → Double thresholding → Edge hysteresis → Binary edge map
Since you are already familiar with the original pipeline, we will now focus on understanding the newly added steps—why they are required and how they improve the final edge map.

2 Non-Maximum-Supression

After applying the gradient filters to the image, two maps are obtained: the *gradient magnitude map* and the *gradient direction map*. For each pixel, Non-Maximum Suppression (NMS) considers a 3×3 neighbourhood and compares the pixel's gradient magnitude with its two neighbouring pixels along the direction of the gradient. If the pixel's magnitude is greater than both of its neighbours, it is retained; otherwise, it is suppressed by setting its value to zero.

Since gradient directions can take any continuous value, they are first quantized into four discrete bins. Each pixel is assigned to a bin based on its gradient angle, and the neighbours are chosen accordingly. The bins are defined as follows:

Direction bins (angle quantization):
Angles are wrapped to $[0°, 180°)$ and then assigned to one of four bins:

Gradient angle range	**Assigned bin (comparison direction)**
$[0°, 22.5°) \cup [157.5°, 180°)$	$0°$ (compare left–right)
$[22.5°, 67.5°)$	$45°$ (compare NE–SW)
$[67.5°, 112.5°)$	$90°$ (compare up–down)
$[112.5°, 157.5°)$	$135°$ (compare NW–SE)

What this does: This step resolves the issue of thick edges by ensuring that only a single pixel is retained across the width of an edge.

Why this works: Smoothing is an essential step in edge detection, as it reduces noise before computing gradients. However, smoothing also spreads a sharp intensity transition over multiple pixels. As a result, when gradient filters are applied, a single edge produces a band of high gradient values rather than a one-pixel-wide response. This band is responsible for the thick edges observed in the output.

To illustrate this, consider a simplified example where an image is represented as an intensity matrix.

Original image (sharp edge):

$$I = \begin{bmatrix} 0 & 0 & 0 & 100 & 100 \\ 0 & 0 & 0 & 100 & 100 \\ 0 & 0 & 0 & 100 & 100 \end{bmatrix}$$

After smoothing:

$$I_{\text{smooth}} = \begin{bmatrix} 0 & 10 & 40 & 70 & 100 \\ 0 & 10 & 40 & 70 & 100 \\ 0 & 10 & 40 & 70 & 100 \end{bmatrix}$$

Gradient magnitude after edge filtering:

$$|G| = \begin{bmatrix} 5 & 20 & 35 & 20 & 5 \\ 5 & 20 & 35 & 20 & 5 \\ 5 & 20 & 35 & 20 & 5 \end{bmatrix}$$

Here, the edge appears as a vertical band of high values instead of a single column. If thresholding is applied directly, this entire band would be classified as an edge, resulting in a thick border.

During Non-Maximum Suppression, assuming the gradient direction is horizontal (0°), each pixel is compared with its left and right neighbours. Only the pixel with the maximum gradient magnitude across this direction is retained:

After Non-Maximum Suppression:

$$|G|_{\text{NMS}} = \begin{bmatrix} 0 & 0 & 35 & 0 & 0 \\ 0 & 0 & 35 & 0 & 0 \\ 0 & 0 & 35 & 0 & 0 \end{bmatrix}$$

Thus, NMS preserves only the strongest response across the edge and suppresses the surrounding weaker responses. Since smoothing cannot be removed from the pipeline without significantly increasing noise, Non-Maximum Suppression is necessary to counteract the edge thickening caused by smoothing, resulting in thin and well-localized edges.

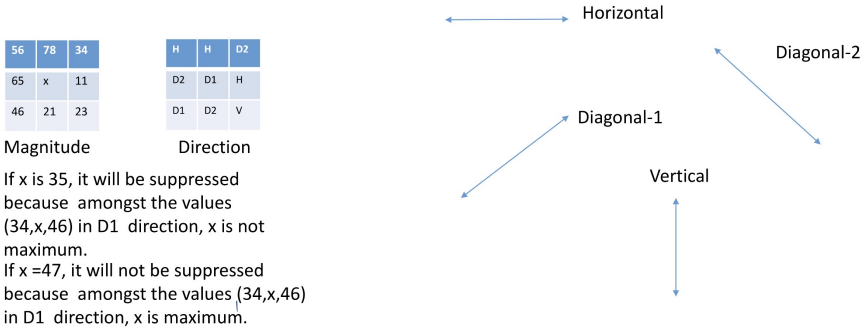

Figure 1: NMS

3 Double thresholding

After Non-Maximum Suppression (NMS), the resulting gradient magnitude image is subjected to *double thresholding*, where two thresholds are defined: a high threshold T_H and a low threshold T_L, with $T_H > T_L$.

A commonly used practical choice is to set the low threshold as a fraction of the high threshold:

$$T_L \approx (0.3\text{–}0.5)\, T_H$$

This helps retain potentially valid weak edges while still suppressing most noise responses.

- Any pixel whose gradient magnitude is greater than or equal to T_H is classified as a **strong edge**.
- Any pixel whose gradient magnitude lies in the range $[T_L, T_H)$ is classified as a **weak edge**.
- All remaining pixels, whose magnitude is less than T_L, are suppressed (set to zero).

Why this is used: Using a single threshold is insufficient. If the threshold is set too high, genuine but low-contrast edges are missed; if it is set too low, noise and texture responses are incorrectly detected as edges. Double thresholding solves this by separating *high-confidence* edges (strong) from *uncertain* edges (weak), allowing weak edges to be validated later.

Why this step is applied after NMS: NMS thins the thick edge responses produced after smoothing and gradient filtering, improving edge localization. If thresholding were applied before NMS, thick edge bands would be retained and the final binary edge map would contain wider, poorly localized edges. Therefore, double thresholding is performed *after* NMS to threshold a thinned, well-localized magnitude map.

How this works (example): Consider the following gradient magnitude values after NMS:

$$|G| = \begin{bmatrix} 0 & 0 & 22 & 0 & 0 \\ 0 & 12 & 18 & 14 & 0 \\ 0 & 0 & 25 & 0 & 0 \end{bmatrix}$$

Let $T_H = 20$ and $T_L = 10$. Then:

- Strong edges: $|G| \geq 20 \quad \Rightarrow$ values 22 and 25

- Weak edges: $10 \leq |G| < 20 \quad \Rightarrow$ values $12, 18, 14$
- Suppressed: $|G| < 10 \quad \Rightarrow$ all remaining entries become 0

At this stage, weak edges are *candidates* and are not yet confirmed as final edges. The final decision on weak edges is made during *edge hysteresis*.

4 Edge Hysterisis

In the *edge hysteresis* step, weak edge pixels are examined based on their connectivity to strong edge pixels. A weak edge pixel is promoted to a strong edge if it is connected to any strong edge pixel through an 8-neighbourhood. This process is applied iteratively: whenever a weak edge is connected to a strong edge, it is reclassified as a strong edge, and the search continues until no further weak edges are connected to strong edges.

After this process is complete, all remaining weak edge pixels that are not connected to any strong edge are suppressed and set to zero. The pixels classified as strong edges after hysteresis form the final *binary edge map*.

Why this works: Real object boundaries form continuous curves rather than isolated pixels. Weak edge responses often correspond to genuine but low-contrast sections of the same boundary due to lighting variations, blur, or texture. If a weak edge pixel is connected to a strong edge pixel, it is highly likely to be part of the same physical edge. Edge hysteresis exploits this continuity by preserving weak edges that are spatially connected to strong edges while eliminating isolated weak responses that are more likely to be noise.

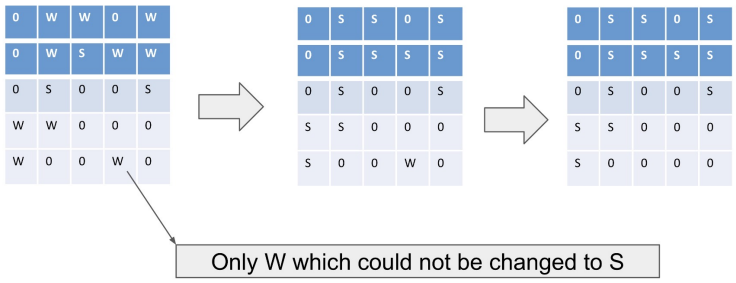

Figure 2: Edge Hysteresis

5 Implementation

In practice, the complete Canny edge detection pipeline—including Gaussian smoothing, gradient computation, Non-Maximum Suppression, double thresholding, and edge hysteresis—is efficiently implemented in OpenCV through the `cv2.Canny()` function.

The primary parameters of the `cv2.Canny()` function are:

- `threshold1`: the lower threshold used for double thresholding. Gradient magnitudes below this value are suppressed.
- `threshold2`: the upper threshold used for double thresholding. Gradient magnitudes above this value are classified as strong edges.

- `apertureSize` (optional): the size of the Sobel kernel used to compute image gradients. It must be an odd integer, typically 3, 5, or 7. A larger aperture increases smoothing but reduces edge localization accuracy.
- `L2gradient` (optional): a boolean flag that specifies how the gradient magnitude is computed. If set to `True`, the more accurate Euclidean norm $\sqrt{G_x^2 + G_y^2}$ is used; otherwise, an approximate $|G_x| + |G_y|$ is used.

A minimal example of applying Canny edge detection using OpenCV is shown below:

```
edges = cv2.Canny(
    image,
    threshold1=50,
    threshold2=150,
    apertureSize=3,
    L2gradient=True
)
```

Here, pixels with gradient magnitude greater than 150 are treated as strong edges, while pixels with magnitude between 50 and 150 are treated as weak edges and retained only if they are connected to strong edges through edge hysteresis. This function produces a binary edge map that closely follows the theoretical Canny pipeline discussed earlier.

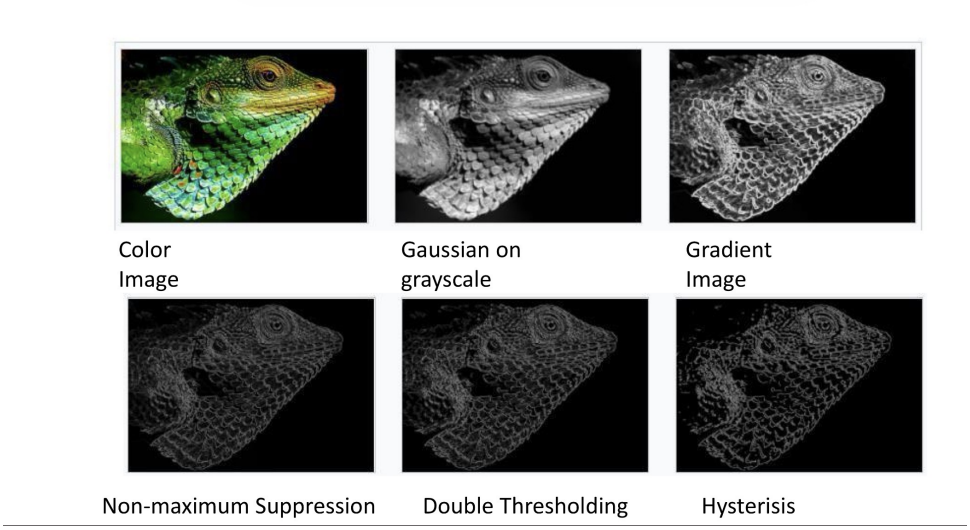

Figure 3: Application of Canny edge detection on an image